# Sonant — AI Voice Agent + PBX Platform

## Full Project Report

> **Brand (go-to-market):** Sonant · **Voice persona:** Nawani · **Internal infra id:** naxter / sampath-ai
> **Tagline:** "Give your business a voice." · **Market:** global **Report date:** 2026-06-18 · **Flagship live deployment:** Holton Hospital (Sinhala/Tamil/English)

---

## 0. Executive summary

Sonant is a **production, self-hosted voice-AI platform** that answers and places **real phone calls**, holds a natural conversation in **Sinhala, Tamil and English**, and **does the actual work** on the call — books a doctor's appointment, takes a product order, makes a table reservation, orders a lab test — writing each result straight into an operational dashboard.

It is built on three pillars:

1. **A real-time voice engine** — an Asterisk PBX bridges live phone audio to **Google Gemini Live** (`gemini-3.1-flash-live-preview`) over a custom TCP audio bridge, with tool-calling so the AI performs grounded transactions instead of just talking.
2. **A multi-industry operations layer** — one config-driven dashboard engine serves per-vertical back-offices (hospital HIS/LIS, reservations, sales/CRM), plus a PBX health/recovery console.
3. **An outbound + e-commerce layer** — automated AI confirmation calls, a single-number broadcast/robocaller, and a public storefront that feeds phone-order confirmations.

**It is genuinely live, not a demo.** On the inspected box there are **63 booked appointments**, **8 sales orders**, **2 lab orders**, **140 customer records**, and **320 logged call sessions** with full transcripts — all from real calls. The first verified real outbound AI call (a sales order confirmation) completed on 2026-06-09.

The architecture is deliberately **vertical-agnostic**: the same engine already spans **finance** (bank branch + FX + lead capture), **hospital**, **sales/e-commerce** and **reservations**, and a new industry is added with config + data, not new plumbing.

---

## 1. What it is — products & surfaces

Four things share one backend and one telephony stack:

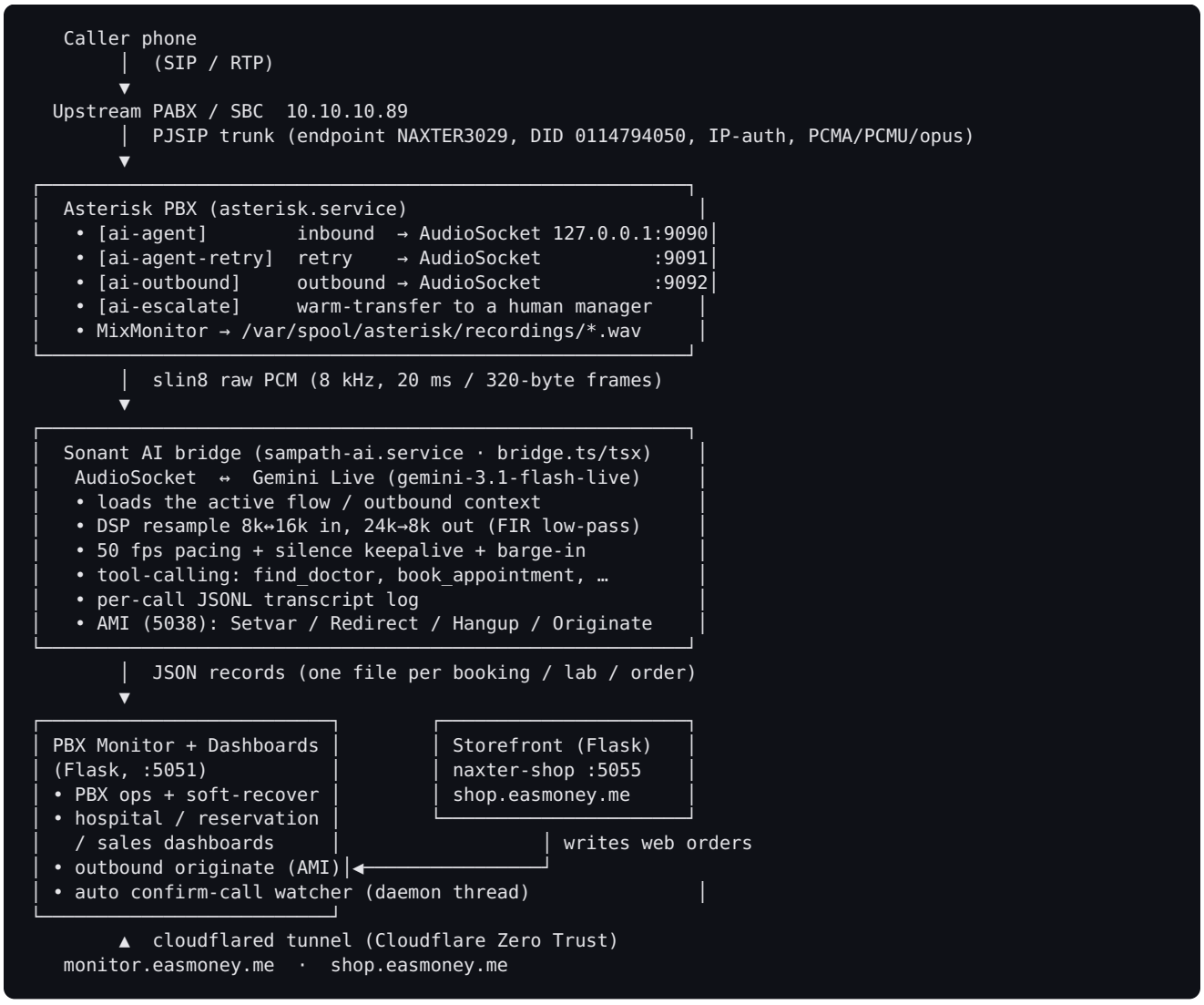
| Product | What it does | Public surface | Service |
|---|---|---|---|
| **Voice Agent** ("Nawani") | Real-time inbound + outbound phone agent; books appointments/orders/ reservations, orders lab tests, confirms & notifies | Phone calls via the SIP trunk | `sampath-ai.service` |
| **PBX Monitor + Dashboards** | | `monitor.easmoney.me` | |

| Product | What it does | Public surface | Service |
|---|---|---|---|
| | PBX ops/recovery console **plus** per-vertical business dashboards (hospital, reservations, sales) | | `pbx-monitor.service` (Flask, :5051) |
| **Storefront** | Public e-commerce store for the Sales vertical; web orders auto-trigger phone confirmation calls | `shop.easmoney.me` | `naxter-shop.service` (Flask, :5055) |
| **Marketing site** | Sonant brand landing page (separate Next.js codebase) | Vercel → `sonant.ai` | — |

The flagship live deployment is **Holton Hospital** (renamed from "Durdans" on 2026-06-09; the internal flow id intentionally stays `durdans`). Nawani takes live Sinhala calls, finds a doctor across 39 branches, books the appointment with a real channelling **queue number** and fee, and the record appears on the hospital dashboard within ~4 seconds.

---

## 2. End-to-end architecture



### Inbound call flow

1. Call hits Asterisk `[ai-agent]` → `AudioSocket(127.0.0.1:9090)`; the dialplan writes the channel name + caller-ID to sidecar files keyed by the call UUID, and starts `MixMonitor` recording.
2. `bridge.ts` accepts the TCP connection, loads the **active flow** (`active-flow.json` → `flows/<id>.json`), and opens a Gemini Live session preloaded with the flow's system prompt + its tool subset.
3. Gemini converses (server-side VAD, barge-in) and calls **tools**; the bridge validates against reference data and writes JSON records atomically.
4. Records appear on the relevant dashboard within ~4 s (the SPA polls every 4 s).

### Outbound call flow (confirmation / notification)

1. Trigger: a dashboard row action **or** the auto-confirm watcher in `app.py`.
2. Flask writes an outbound context file `/var/lib/sampath-ai/outbound/<uuid>.json` and **AMI-originates** `PJSIP/<phone>@pabx` into `[ai-outbound]` (caller-ID `Naxter <0114794050>`).
3. On answer, `[ai-outbound]` → `AudioSocket(:9092)`; the bridge builds a confirm/notify persona from the context and runs the call, writing the outcome back onto the record.

---

## 3. Technology stack (complete)

**Voice agent (`sampath-ai.service`)** - **Runtime:** Node.js via `tsx` (TypeScript executed directly, no build step) - **Entry point:** `bridge.ts` (~80 KB, the core engine) - **Brain:** Google **Gemini Live** — `gemini-3.1-flash-live-preview` via raw WebSocket (`ws`), BidiGenerateContent v1beta; AUDIO response modality; input **and** output audio transcription enabled - **Voices:** Gemini prebuilt voices (default **Aoede**; 14 available e.g. Zephyr/Kore/Leda/Despina) - **Telephony I/O:** Asterisk **AudioSocket** protocol (raw `slin8` 8 kHz 16-bit PCM) over TCP - **Audio DSP (custom):** stateful linear-interpolation upsampler (8k→16k) and a **31-tap Hamming-windowed sinc FIR low-pass** (fc≈3700 Hz) + 3:1 decimator (24k→8k) to kill sibilant aliasing - **Control plane:** Asterisk **AMI** over TCP 5038 (Login/Setvar/Redirect/Hangup/Originate) - **Key deps:** `@google/genai`, `ws`, `uuid`, `dotenv`; `perf_hooks` event-loop watchdog

**Dashboards / PBX monitor (`pbx-monitor.service`)** - **Framework: Flask** (Python), single Werkzeug process, bind `127.0.0.1:5051` - **Frontend:** Jinja2 (`base.html` + `industry.html`) + a vanilla-JS SPA engine (`industry-engine.js`) + **Chart.js** + lucide icons - **Auth:** Werkzeug password hashing; per-user roles + **per-user dashboard access** (`auth.json`) - **TTS for broadcast: AWS Polly** (boto3), neural voice (default "Matthew"), cached by SHA-256 - **Storage:** flat **JSON files** on disk — no database

**Storefront (`naxter-shop.service`)** - **Framework:** single-file Flask (`shop/shop.py`, ~176 lines) + `store.html`, bind `127.0.0.1:5055` - Server-side hardened checkout (price recompute, stock lock, rate limit, HTML-escape, payload cap) - Frontend: Tailwind via CDN, vanilla JS, localStorage cart

**Telephony / infra** - **PBX:** Asterisk (`asterisk.service`); **PJSIP** trunk to upstream PABX/SBC `10.10.10.89` (IP-auth, PCMA/PCMU/opus) - **Ingress: Cloudflare Tunnel** (`cloudflared.service`) — `monitor.easmoney.me`, `shop.easmoney.me` - **SIP capture:** `sip-capture.service` (rolling pcap for diagnostics) - **Recovery:** `voip-probe.timer` (10 s ping probe), snapshot tooling, soft-recover button - **OS / process mgmt:** Linux + systemd; service user `asterisk`; Tailscale present

**Marketing site (separate repo `/home/horapusa/sonant-landing`)** - **Next.js 14** (App Router) + **React 18** + **TypeScript 5** + **Tailwind 3** + **Framer Motion 11** - Fonts: Space Grotesk / Inter / JetBrains Mono; dark theme `#09090b`, accent `#ff5d3b`; deploy → Vercel `sonant.ai`

## 4. The voice agent engine (deep dive)

`bridge.ts` is a single Node/TS TCP server that is the heart of the platform. Per call it does the following.

**Transport — AudioSocket framing.** Parses the binary AudioSocket protocol (1-byte type, 2-byte big-endian length, payload): `0x01` = call UUID, `0x10` = audio (slin8), `0x00` = hangup, `0xFF` = error. Three listeners: **:9090** inbound, **:9091** retry (after a failed human transfer), **:9092** outbound.

**Brain — Gemini Live.** Opens a WSS session, sends a setup message with the model, voice, AUDIO modality, a fully-assembled system instruction, the flow's tool declarations, and **both** input + output transcription. Relies on Gemini's **server-side VAD** for turn-taking; `serverContent.interrupted` drives **barge-in** (the bridge instantly clears its outbound buffer so the agent stops talking the moment the caller speaks).

**Audio quality.** Gemini emits TTS in bursts (~5 s of audio in ~2 s). The bridge runs a 20 ms pace timer that drains audio to Asterisk at exactly **50 fps**, keeping Asterisk's queue ~1 frame deep so barge-in is near-instant. Two earlier production bugs were fixed here: - **Calls dying ~10 s after greeting** → Asterisk's `app_audiosocket` drops the channel after **2000 ms** of silence; fix = emit a 20 ms zero "silence keepalive" frame whenever idle >500 ms. - **Buzzy "grr" on fricatives** → an IIR low-pass let 4–8 kHz aliasing through (8,638 aliasing artifacts measured in one 153 s call); fix = the 31-tap FIR decimator.

**Grounded conversation.** A 14-tool registry (next section) lets Gemini perform real transactions. Strict anti-fabrication: the agent must **never invent** a doctor, fee, reference number, product or stock — those values come only from tool returns. Booking tools reject placeholder names/phones ( `validName` / `validPhone` ) and return `need_details` if called before the agent has actually collected the data.

**Multilingual handling.** Per-turn language detection by Unicode script (Sinhala U+0D80–0DFF, Tamil U+0B80–0BFF) plus a romanized-Sinhala regex (Gemini latinizes spoken Sinhala). Tallies pick the call language (Sinhala is the house default); outbound calls inherit the language the customer used inbound. Reference numbers/names/dates are treated as **data**, so the agent doesn't flip to English to read out a `HOL-AP-######` .

**Phone capture.** The caller-ID is injected into the prompt **spaced digit-by-digit**; the agent reads it back one digit at a time and confirms any alternate number the same way — a deliberate workaround for STT errors on grouped Sinhala numerals (e.g. "□□□□ □□" → 38). `normalizeLkPhone` canonicalises everything to `0XXXXXXXXX` .

**Escalation & hangup.** `request_human_transfer` resolves a manager number by category, waits for the agent to finish its hold line, then AMI-Setvar `AI_MGR_NUMBER` + AMI-Redirect to `[ai-escalate]` . `end_call` waits for the goodbye to drain, then AMI-Hangup. A 3-phase "wait for agent to finish" guard prevents cutting off speech.

**Observability.** Every call appends a JSONL event log to `/var/lib/sampath-ai/sessions/<uuid>.jsonl` — `session_open` , `gemini_ready` , `trigger_chosen` , `transcript {role,text}` , `tool_call` , `booking_created` , `escalation_requested` , `ami_*` , `session_close` , errors. An **event-loop watchdog** warns when Node p99 stalls >200 ms (a stall would break the keepalive and drop the call).

**Working-hours routing.** Per-day HH:MM schedules (Asia/Colombo) decide out-of-hours behaviour: greet / transfer-to-human / polite-hangup.

> **Known engine limitation:** there is **no Gemini reconnect** — a mid-call WebSocket drop ends the call. There is no max-concurrent-call cap or per-call duration cap, and the `GEMINI_API_KEY` is passed in the WSS query string.

---

## 5. Telephony / SIP / PBX layer

- **Trunk:** a single PJSIP trunk ( `pabx` ) to the upstream SBC/PABX at **10.10.10.89:5060**, endpoint **NAXTER3029**, DID **0114794050** (Colombo), IP-based auth (no digest), codecs PCMA/PCMU/opus over UDP. Tenant: `NAXTER_AI_SOLUTIONS_PVT_LTD_DEVINUWARA` .
- **Dialplan contexts:** `[ai-agent]` (9090), `[ai-agent-retry]` (9091), `[ai-outbound]` (9092), `[ai-escalate]` (manager transfer), and `outbound-on-answer-*` (broadcast/IVR playback). Per-call channel + caller-ID sidecar files ( `<uuid>.chan` / `.cid` ) bridge dialplan ↔ Node.
- **Caller-ID capture** is applied (by an in-place awk patch) **only** to inbound contexts — on outbound calls the caller-ID is our own line.
- **Security at the dialplan:** the outbound UUID is sanitised with `FILTER(0-9a-f-,…)` before it reaches `AudioSocket` / `System()` , preventing dialplan/shell injection via the AMI variable.

> **Source-control caveat:** only `asterisk/ai-outbound.conf` is checked into the staging repo. The live **inbound** dialplan and `pjsip.conf` exist only in `/etc/asterisk` on the box and are mutated in place by the installer scripts (awk/python with marker guards). Losing the box loses the inbound dialplan source — a backup gap worth closing.

---

## 6. Agent tools & "prompts as code"

### 6.1 Tool catalogue (Gemini function-calling)

Schemas live in `flows/patches/gemini-live.ts` (a single `TOOL_REGISTRY` ); handlers live in `bridge.ts` ( `handleToolCall` ). A flow exposes only the subset listed in its `tools_enabled[]` .

| Tool | Vertical | Effect |
|---|---|---|
| `find_doctor` | Hospital | Symptom/specialty → doctor + branch (relaxing fallback chain; never returns empty) |
| `book_appointment` | Hospital | Writes appointment; returns `HOL-AP-######` , `queue_no` , session time, fee; upserts patient MRN |
| `order_lab_test` | Hospital | Creates a lab order + accession from the test catalogue |
| `book_reservation` | Reservations | Table booking with party cap + auto deposit |
| `find_product` / `place_order` | Sales | Catalogue lookup (live stock) + phone order (COD/Card) |
| `confirm_order` | Sales (outbound) | Flip order → confirmed / cancelled / reschedule |
| `call_outcome` | Hospital/Sales (outbound) | Record outcome (e.g. shipped notification, critical-lab ack) |
| `request_human_transfer` | All | Warm-transfer to a human (hospital → `0740690110` ) |

| Tool | Vertical | Effect |
|---|---|---|
| `find_sampath_branch` | Finance | Locate a bank branch |
| `get_exchange_rates` | Finance | Live FX rates |
| `save_customer_info` | All | Persist captured fields (per-flow field hints) |
| `end_call` | All | Graceful hangup |

### 6.2 Prompts & flows as code

- A **flow** is a JSON file ( `flows/<id>.json` ) = persona ( `system_prompt` + voice + greeting) + admin-editable `custom_instructions` + `transfer_rules[]` + `tools_enabled[]` + `working_hours` + `record_calls` .
- The **hospital** flow is managed as code: `setup-flows.py` rewrites its `system_prompt` from `flows/prompts/hospital-appointment.system.md` and **sets** (not appends) its `custom_instructions` (HOSP + PAY + LAB + NO-DISCLAIMER blocks) on **every deploy** — this exists to kill a real past bug where stale guidance told the agent to fabricate a "HOL-AP" number so bookings never reached the dashboard.
- Reservations and Sales flows are seeded **inactive**; activating a vertical is an explicit admin action that repoints `active-flow.json` (no service restart — the bridge re-reads it on the next call).
- Effective compliance technique for this model: explicit **WRONG / RIGHT** examples in the prompt (e.g. "ask the branch first" and the hard no-medical-disclaimer rule).
- The Flows admin page can **AI-generate** prompts and a visual flow diagram (React Flow) via Gemini 2.5 Pro; presets are read-only and must be cloned to edit.

---

## 7. Industry verticals

The platform ships verticals at two levels: **full dashboards** (hospital, reservations, sales) and **voice-only flow presets** (finance/bank, real-estate, software helpdesk). The user's "for now" set — **finance, hospital, sales, reservations** — is all present.

### 7.1 Hospital (flagship — a working mini HIS/LIS)

**Reference data scale:** 39 branches across all 24 districts, **94 doctors** (D001–D094), 16 specialties, a 13-test lab catalogue with analyte units/ranges and critical thresholds. A deterministic generator ( `generate-hospital-refdata.py` ) guarantees every branch has GP cover.

**Appointments:** `booked → confirmed → checked_in → completed` (or `cancelled` / `no_show` ), with Sri-Lankan **channelling semantics** — a `queue_no` scoped to doctor+branch+date and a `session_time` picked from the doctor's first slot in the requested day-part. Created identically by the voice agent or by staff on the dashboard.

**Lab (LIS):** a proper 8-state pipeline — `ordered → collected (accession) → received → in_process → resulted → verified → delivered` (+ `rejected` ). **Server-authoritative analyte flagging:** result entry recomputes units/ranges from the catalogue server-side, flagging `H` / `L` / `critical` (a client can't spoof a flag). Critical results raise a "to call" KPI.

**Patients:** a registry deduped by phone, keyed by auto-generated **MRN-#####**.

**Billing & reports (client-side):** derived consultation + lab line items, paid/unpaid tracking, and analytics — confirmation rate, no-show rate, lab volume, revenue split (consult vs lab), and **average lab turnaround time (TAT)**.

**Outbound:** manual appointment confirm/remind calls and **critical-lab callbacks** (the agent urges prompt follow-up and records the acknowledgement, but never gives medical advice).

### 7.2 Reservations

Table bookings with party size, date/time, **seating area** and branch; deposit auto-computed as `party × Rs 1500/guest` (server-side). Lifecycle `pending → confirmed → seated → completed` (+ cancelled/waitlist/refunded), deposits tracked paid/refunded. Caveat: the reservations "call to confirm" queue is currently a **client-side simulation** — unlike sales/hospital there is no real outbound endpoint wired yet (clear next step).

### 7.3 Sales / e-commerce (most built-out)

- **Leads (CRM funnel):** `new → contacted → qualified → converted/lost`, with "convert to order".
- **Orders from three channels:** inbound AI call ( `place_order` ), the public **storefront** ( `shop.py` ), and staff entry — all landing in the same store, indistinguishable.
- **Order lifecycle:** `pending → confirmed → processing → shipped → delivered` (+ no_answer/cancelled/refunded).
- **Live stock** is computed everywhere as `initial stock − qty in non-cancelled orders` (never mutated), so storefront, agent and dashboard always agree; the shop re-checks under a lock at write time to prevent overselling.
- **Automated outbound confirm calls** the moment a non-AI order lands (see §9).

### 7.4 Finance (voice-flow level)

The product's origin (Sampath Bank): a flow preset exposing `find_sampath_branch` + `get_exchange_rates` + `save_customer_info` (lead capture), backed by a live branch/FX-rate cache. There is no dedicated finance dashboard yet — adding one is pure config (see §13). The marketing site already positions "Finance & services" as a first-class vertical.

---

## 8. Call recordings & transcripts

**Transcripts — fully implemented for every call.** Each call (inbound and outbound) produces a structured JSONL log at `/var/lib/sampath-ai/sessions/<uuid>.jsonl` with timestamped events including turn-by-turn `transcript {role: agent|user, text}` from Gemini's input/output transcription. **320 such session logs** exist on the live box. The dashboard renders these as chat bubbles per call, alongside captured customer fields.

**Audio recording — implemented for inbound + manual calls, asymmetric for AI outbound.** - The live `[ai-agent]` dialplan runs `MixMonitor` on every answered inbound AI call → `/var/spool/asterisk/recordings/<ts>_<from>_<to>_<uuid>.wav`. - All manual `outbound-on-answer-*` (Broadcast / Make-Call) contexts also record. - `[ai-outbound]` **(automated AI confirm/notify calls) has no** `MixMonitor` → those calls are **transcript-only, no audio**. Adding `MixMonitor` there is an obvious, low-effort upgrade. - A **Recordings dashboard** lists WAVs (parsed metadata) with inline **HTML5 playback**, download and delete; per-call lookup correlates audio ↔ transcript by the call UUID.

**Caveats worth noting:** - The `MixMonitor` lines live only in the on-box dialplan (not version-controlled in staging); a per-flow `record_calls` toggle exists but the inbound dialplan records regardless, so the toggle is effectively vestigial today. - There is **no recordings retention/prune job** (only old diagnostic snapshots are pruned at 14 days) — disk growth is unmanaged. - No server-side transcription search, no PII/PCI redaction, no bulk export.

---

## 9. Outbound calling & "broadcasting"

There are **three** outbound mechanisms today, all built on one primitive ( `_originate_outbound` → AMI-Originate `PJSIP/<num>@pabx` ):

1. **Automated AI confirm-call watcher** — a single flock-guarded daemon thread sweeps `sales/orders` every 6 s; each new `pending` order (that didn't come from an AI call) is **claimed** ( `auto_call_at` stamp written before dialling, so it can never double-ring), then an AI confirmation call is placed. Gated by `auto_confirm_call` , an `auto_call_hours` window (default 08:00–21:00), and the `confirm_test_number` safety switch. **This is live to real buyers** (verified 2026-06-09). It is **event-driven 1:1**, not a campaign.
2. **Manual outbound AI calls** — dashboard buttons for appointment confirm/remind, lab-result/critical callbacks, order confirm and shipped-notification calls (auth + role-gated).
3. **Broadcast / Make-Call page** — a **single-number** robocaller: dial one number, play a stored WAV **or** type a script synthesised by **AWS Polly TTS**, optionally IVR-forward to a manager.

> **"Broadcasting" is a single-call tool today, not mass dialing.** There are **no recipient lists, no CSV upload, no campaign object, no scheduler, no retry-on-no-answer, and no DNC/opt-out list**. The per-call engine is solid and duplicate-safe; what's missing is the orchestration layer on top — that is the single biggest outbound roadmap item (see §15).

---

## 10. Dashboards & operations

**Config-driven engine.** One generic SPA engine ( `industry-engine.js` ) renders every vertical from a declarative config ( `static/data/<vertical>.js` → `window.INDUSTRY` ): tabs, KPIs, status maps, action buttons, derived collections, charts, reports, CSV export. Live verticals fetch `/api/dash/<vertical>/<collection>` every 4 s with optimistic insert/update queuing; demo verticals fall back to localStorage.

**Auth & access.** Werkzeug-hashed logins; users have roles and a **per-user dashboard allowlist** ( `auth.json` ); admins see all. State-changing call actions require a `call` permission.

**PBX ops & recovery console ( `/trunk` ).** - Live SBC heartbeat + SIP/RTP tails (parsed from the rolling pcap, 2 s refresh). - **"Recover trunk"** (admin only): a soft recovery — snapshot → restart `sip-trunk-route` → restart `sampath-ai` → re-qualify the trunk AOR → only reload Asterisk if still dead. **Never reboots, never restarts Asterisk.** (Designed after the original "No heartbeat" panic-reboots were traced to a false-alarm bug: the panel was reading a non-existent pcap path.) - **Incident snapshots** ( `pjsip` state, routes, pings, journals, pcap copy) captured before any restart, browsable in the UI, pruned at 14 days. - **Connectivity probe** every 10 s appends gateway/SBC ping CSV to `probe.log` .

---

## 11. Storefront (Naxter Store)

A public, unauthenticated Flask shop ( `shop/shop.py` ) at `shop.easmoney.me` , sharing the **same** product catalogue and order store as the agent and dashboard. A web checkout appears on the staff dashboard in ~4 s and immediately enters the auto-confirm-call queue.

**Checkout hardening (all server-side):** - **Price recompute** — client-submitted prices are ignored; line prices and total are recomputed from the catalogue, so a tampered cart cannot change what's

charged/recorded. - **Stock lock** — two-phase availability check with a fresh re-check inside a lock at write time (no overselling the last unit); atomic `tmp + rename` writes. - **HTML-escape + clamp** — every customer field is `html.escape()`d, control-char-stripped and length-clamped before storage (protects the dashboard that later renders it). - **Rate limit** — 6 orders / IP / 60 s (Cloudflare/XFF-aware), 64 KiB payload cap. - **Least privilege** — binds `127.0.0.1` only (public solely via Cloudflare Tunnel), runs as unprivileged `asterisk`.

Caveat: no payment gateway ("Card" is a label), and **no phone-number ownership/OTP check** — a placed order triggers a real outbound call to whatever number is entered, so a CAPTCHA / number-verification layer is advisable before scaling.

---

## 12. Data model & storage

Everything is **flat JSON, one file per record**, under `/var/lib/sampath-ai/` (atomic `tmp + rename` everywhere; the dashboard polls the directories):

```
active-flow.json                          # which flow is live (durdans / reservations / sales / …)
flows/<id>.json                           # flow config (prompt + custom_instructions + tools + working_ho
refdata/hospital.json                     # 39 branches / 24 districts / 16 specialties / 94 doctors + 13
refdata/{reservations,sales}.json         # catalogues + outbound config (confirm_test_number, auto_call_h
bookings/hospital/{appointments,labs,patients}/<id>.json
bookings/reservations/reservations/<id>.json
bookings/sales/{orders,leads}/<id>.json
outbound/<uuid>.json                      # outbound call context (kind, customer, summary, language…)
sessions/<uuid>.jsonl                     # per-call transcript + event log
channels/<uuid>.{chan,cid}                # dialplan↔bridge sidecars (created/removed per call)
customers/<key>.json                      # captured fields from save_customer_info
voice-samples/*.wav                       # voice-clone / TTS samples
/var/spool/asterisk/recordings/*.wav      # call audio (on-box)
/var/log/asterisk/cdr-csv/Master.csv      # CDR
```

**Live data on the inspected box (real usage):** 63 appointments · 2 lab orders · 8 sales orders · 140 customers · 320 call sessions · active flow `durdans` · live voice `Aoede`.

---

## 13. Cross-industry extensibility — adding a new vertical

The engine is genuinely vertical-agnostic. To add, say, a **finance** dashboard vertical (loan applications + payment reminders) requires **no engine changes**:

1. **`app.py`** — add a `DASHBOARDS` entry, an `AGENT_MODES` mapping to a flow, `DASH_COLLECTIONS['finance']` + record-id prefixes, and any server-side enrichment keyed on the vertical (mirroring the hospital doctor-fee or reservations-deposit blocks).
2. **`refdata/finance.json`** — the catalogue (products/rate tiers) + the same `confirm_test_number` / `auto_confirm_call` / `auto_call_hours` keys to reuse the outbound rails.
3. **`static/data/finance.js`** — a declarative `window.INDUSTRY` (tabs/KPIs/statuses/actions) identical in shape to `sales.js`.
4. **`bridge.ts`** — add the new agent tools (`find_loan`, `submit_application`, `confirm_payment`) writing to `bookings/finance/…`, plus an outbound persona for any new "kind".
5. **Reuse the rest verbatim** — outbound origination, the `confirm_test_number`/hours guards, the `[ai-outbound]` dialplan, recordings, transcripts, auth, and live polling all already take the vertical as a parameter.

The one hardcoded spot is the **auto-confirm watcher** (sales-only today) — generalising it to iterate configured verticals is the single cleanest refactor for true multi-vertical outbound automation.

The universal, reusable patterns across all verticals: (1) **tool-grounded anti-fabrication**; (2) **collect → confirm (read back) → commit**; (3) **ask-the-disambiguator-first** (branch/area/product); (4) **one active persona switchable by a single pointer file, no restart**; (5) **category-based human escalation via a per-call channel variable**.

---

## 14. Deployment, infra & recovery

- **Source of truth / staging:** `/home/horapusa/voip-recovery-staging/` (owned by `horapusa`). Edit in staging, then run an `install-*.sh` script which **backs up** the live file (`*.bak-<tag>-<ts>`), copies into `/opt/...` as user `asterisk`, and restarts only the affected unit.
- `sudo` **requires a password** (not passwordless) — deploys are run by the human, never automatically. The only NOPASSWD grants are the four recovery actions + `snapshot.sh`.
- **Pre-deploy validation:** `esbuild` / `tsc` transform of the TS, Python `ast.parse`, JSON parse, a `setup-flows.py` dry-run, and Flask endpoint checks.

| Script | Deploys | Restarts |
|---|---|---|
| `install-agents.sh` | `bridge.ts`, `gemini-live.ts`, all refdata, booking dirs; runs `setup-flows.py` | `sampath-ai` |
| `install-dashboards.sh` | `app.py`, engine, all data files, `industry.html` | `pbx-monitor` |
| `install-shop.sh` | `shop/` storefront (+ systemd unit) | `naxter-shop` |
| `install-outbound.sh` | `[ai-outbound]` dialplan + `dialplan reload` | Asterisk |
| `install-callerid.sh` | in-place caller-ID capture patch | Asterisk |
| `install-flows.sh` / `-v2.sh` | multi-agent flow builder, escalation patch, customers/playground/working-hours UI | `pbx-monitor` + `sampath-ai` |
| `install.sh` / `uninstall.sh` | soft-recovery toolchain (probe, snapshots, sudoers, logrotate, cron) | `pbx-monitor` + `sampath-ai` |

`generate-hospital-refdata.py` regenerates the 94-doctor directory; `merge-duplicate-patients.js` reconciles `+94`-vs-`0` duplicate patient records.

---

## 15. Security & safety

**Strengths** - Storefront checkout is well-hardened (price recompute, stock lock, escaping, rate-limit, payload cap, least-privilege bind). - Outbound dialing is **duplicate-safe** (claim-before-dial), **time-windowed**, and has a per-vertical `confirm_test_number` kill-switch that reroutes every outbound call to a test number while testing. - Server-authoritative computed fields (lab flags, deposits, queue numbers) can't be spoofed by a client; PATCH routes whitelist mutable fields (money/phone are excluded). - Path-traversal guards on record/lab-action/outbound/snapshot routes; UUID sanitisation in the dialplan; conservative no-reboot recovery.

**Gaps to address before scaling** - **No DNC/opt-out list, no consent ledger, no global concurrency/rate cap** on outbound (the only brakes are `confirm_test_number` + an hours window) — important for telecom compliance, especially for finance/healthcare. - **No phone-number verification** on the public storefront → potential robocall-amplifier risk; add CAPTCHA/OTP. - `GEMINI_API_KEY` is sent in the WSS query string; AMI creds fail silently to empty. - Inbound dialplan + `pjsip.conf` are not backed up to staging. - No call-audio retention policy; AI outbound calls aren't audio-recorded (only transcribed) — a regulatory consideration where full recording is required.

---

## 16. Live status

- ✓ **Hospital agent live** taking real Sinhala calls; appointments + labs land on the dashboard (63 appts / 2 labs recorded).
- ✓ **First verified real outbound AI call** (sales auto confirm-call) end-to-end on **2026-06-09**; sales auto-confirm is **live to real buyers**.
- ✓ **Storefront deployed and healthy**; web orders flow to dashboard + trigger confirm calls.
- ✓ **320 call sessions** transcribed; **140 customers** captured.
- → **Not built yet:** mass/campaign outbound (lists + scheduler + retry/backoff + DNC), reservations real outbound calling, finance dashboard, auto-retry on no-answer, payment gateway, recordings retention, server-side transcript search/redaction, multi-tenant per-DID routing, pharmacy/IPD/insurance/PDF-report/LOINC for the hospital HIS.

---

## 17. Roadmap / future (prioritised)

**Near-term, high-value, low-effort** 1. Add `MixMonitor` to `[ai-outbound]` so AI confirm/notify calls are audio-recorded too. 2. Add a recordings **retention/prune** job + storage quota. 3. Wire **reservations real outbound** confirm calls (mirror sales) and an auto-confirm watcher. 4. Back up the on-box inbound dialplan + `pjsip.conf` into staging (close the source-control gap). 5. Move `GEMINI_API_KEY` to header/ephemeral-token auth.

**Medium-term — the campaign engine (biggest single feature)** 6. A true **outbound campaign layer** on top of `_originate_outbound`: recipient lists / CSV upload, scheduler, pacing + concurrency cap, **retry-on-no-answer with backoff**, **DNC/opt-out + consent ledger**, and per-campaign analytics. 7. Generalise the **auto-confirm watcher** to iterate configured verticals (true multi-vertical automation). 8. **Finance dashboard** vertical + KYC/collections/payment-reminder personas. 9. Phone-number **OTP/verification** + CAPTCHA on the storefront. 10. Gemini **session reconnect/ resume** + a max-concurrent-call cap + per-call duration cap.

**Longer-term — platform & product** 11. Move from flat JSON to a real datastore (the engine already abstracts the store) for indexing, history, date-range reports, and multi-tenant scale. 12. **Multi-tenant / per-DID routing** so one box can run several verticals simultaneously (today one active flow at a time). 13. Server-side **transcript search, summaries, sentiment, PII/PCI redaction**, bulk export — already promised in the marketing copy. 14. CRM/calendar/webhook **integrations** (Google Calendar, HubSpot/Salesforce, WhatsApp, Zapier) as advertised on the landing page. 15. Deeper hospital HIS: pharmacy, IPD/wards, insurance, PDF report export, LOINC coding, age/sex-specific lab ranges, microbiology, no-show auto-sweep.

---

## 18. Brand & go-to-market

**Sonant** is the global consumer brand (formerly naxter/ryzera); the AI persona is **Nawani**. The Next.js landing page (→ Vercel `sonant.ai`) pitches it **horizontally**: "Give your business a voice that never sleeps … answers every inbound call, places outbound calls at scale, and books appointments, reservations and orders — in 40+ languages, around the clock. Billed by the minute."

- **Six headline features:** Inbound answering · Outbound at scale · **Actions** (writes real bookings into your system, not just a transcript) · human-sounding multilingual voice · every call on the record (transcripts/summaries/analytics) · integrations (SIP/PSTN, calendars, CRMs, webhooks, warm transfer).
- **Six target industries:** Healthcare · Restaurants & hospitality · Retail & e-commerce · Real estate · Logistics · Finance & services — overlapping the backend's hospital/reservations/sales/finance verticals.
- **Headline stats (marketing):** `<800 ms` latency · `40+` languages · `99.9%` uptime · `24/7`.
- **Pricing (placeholder):** per-minute (inbound $0.09 / outbound $0.12 / number $2/mo) with tiers Starter (free PAYG) / Growth ($299/mo) / Scale (custom) and a live cost estimator.

> Pre-launch cleanup flagged on the site: CTAs/footer links are `#` placeholders, pricing is placeholder, and one **stale** `DUR-AP-482917` **booking code** + a Sri-Lanka/cardiology hero demo survive from the Holton origin and should be made brand-neutral; quantified claims (latency/uptime/language count, testimonial outcomes) need substantiation.

---

## 19. One-paragraph "what makes it notable"

Most "AI voice" demos either play a recording or transcribe a call. Sonant **does the transaction on the line** — it disambiguates ("which branch?"), captures and reads back a phone number digit-by-digit, looks up a real doctor, computes a real channelling queue number and fee, writes an atomic record, and that record is on a staff dashboard four seconds later — in Sinhala, on a self-hosted Asterisk trunk, with the same engine reskinned for finance, hospitals, restaurants and e-commerce by **config alone**. The engineering depth is in the unglamorous places that make a phone agent actually usable: FIR anti-aliasing for clean speech, silence-keepalive pacing to survive Asterisk's timeout, barge-in, server-authoritative anti-fabrication, claim-before-dial outbound safety, and a no-reboot recovery console.

---

Report compiled 2026-06-18 from the live system and the *`voip-recovery-staging`* source tree. Authoritative architecture reference: *`voip-recovery-staging/PROJECT.md`*.